SECURITY TOOL / ENGLISH TECHNICAL GUIDE

Jev for Web Security

Detect the trial-and-error behind AI-agent-driven web attacks — not just individual requests.

Prototype v0.1.0 • October 2026 • Independent research software

1. Motivation: attacks are sequences, not isolated events

AI-assisted operators can automate tasks that previously required substantial manual effort: identifying endpoints, checking responses, retrying failed requests, and changing tactics. An individual HTTP request may contain no obvious exploit payload; the broader sequence can nonetheless reveal persistent probing or a shift in the attacker's approach.

This creates a practical detection challenge. Request-level checks remain essential for SQL injection, cross-site scripting, and other recognizable patterns, while advanced WAF and bot-management products may also correlate events. Our tool addresses a narrower complementary problem: helping an analyst inspect how one actor's requests evolve over time, especially after errors or access denials.

Importantly, the tool does **not** claim to identify AI use from Web logs alone. Humans, scripts, and AI agents may produce similar traces. It instead looks for observable patterns consistent with iterative, adaptive exploration. This distinction keeps the output useful even when the attacker's tooling is unknown.

The design idea

Jev for Web Security reconstructs short, ordered request histories from server-visible metadata and converts them into bounded, privacy-conscious observations. Ordinary Python code calculates frequencies, response distributions, and changes in routes or methods. Jev, a System One model from TypeSafe, is then used as a structured behavioral classifier. The same workflow also runs with a transparent offline rule baseline when no Jev credentials are available.

What a reviewer should take away

Question	Answer in the current prototype
What is analyzed?	Ordered HTTP request/response metadata grouped by server-derived actor or source-IP fallback.
Why a window?	It makes retries and behavior changes visible while keeping inputs bounded and causal.
What is the result?	A behavior label, review indication, computed evidence, and exportable JSON for analyst review.
What is not claimed?	AI attribution, confirmed compromise, production blocking, or benchmarked detection superiority.

2. System design: from web logs to a behavioral assessment

The tool is deliberately deployed outside the live request path. It accepts normalized Web/API JSONL metadata or a documented subset of ModSecurity v3 JSON Lines. This avoids introducing inline blocking into the first prototype and makes it possible to replay exactly the same observations across alternative detection settings.

Each record is minimized before analysis. Requests are grouped into entity-specific sequences, and a sliding window plus bounded historical summary is constructed before the decision backend is called. The architecture below separates conventional counting from model inference.

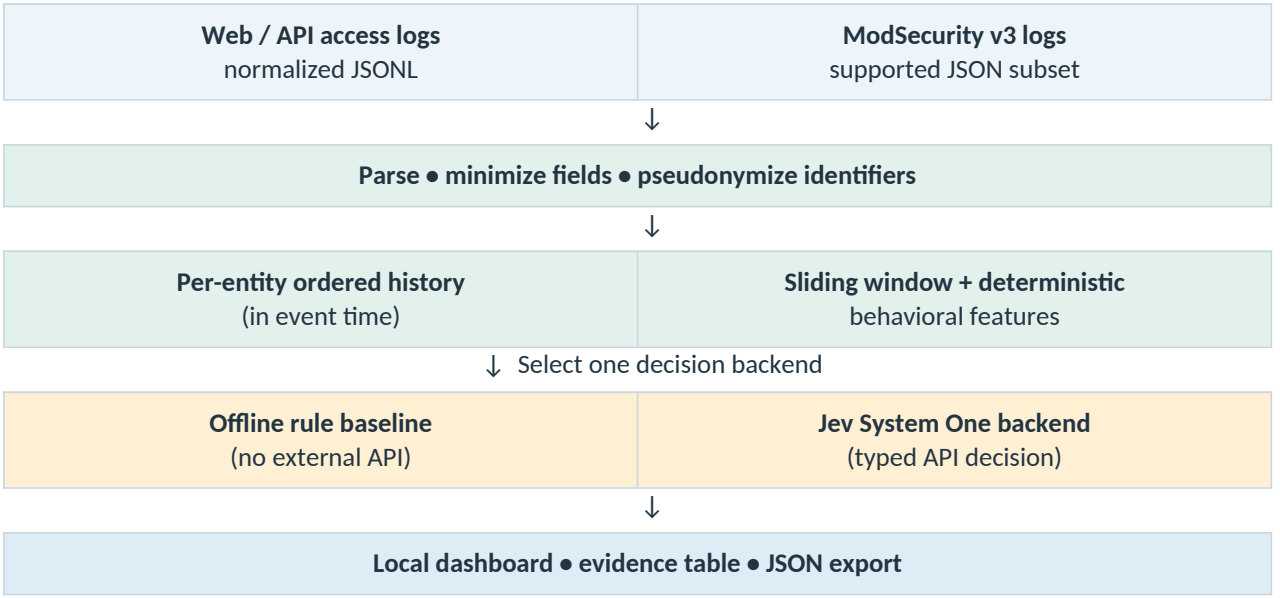


Figure 1. Editable architecture diagram. The independent rule and Jev modes share the same log-processing pipeline; rule scores must not be mistaken for Jev predictions.

What Jev contributes—and what stays in Python

The System One backend receives the bounded sequence and compact numerical summary as a state to classify, rather than an unbounded raw HTTP transcript. It returns structured assessments intended for review. Python is responsible for exact counts and feature construction, while the dashboard displays the server-derived observations that underlie the decision.

Conventional processing	Jev model processing
Request counts, response-code mix, route changes, time gaps	Behavior classification across the ordered observations
Actor partitioning, timestamp ordering, history caps	Review-oriented assessment for the current window
Display of factual log evidence	Structured output; no invented “chain of reasoning”

What Jev receives

The minimized model input contains an ordered sequence of HTTP method, normalized route, response status and time gaps, along with Python-computed counts and recent historical summaries. HTTP bodies, headers and query strings are excluded by default. This makes the model input inspectable and bounded rather than an unfiltered log transcript.

A Jev API client is implemented, but this release has not yet been validated with live credentials. That qualification is intentionally visible in the UI and documentation.

3. Sliding Window: why 20 requests, and what is preserved

A single event rarely shows trial-and-error. The tool therefore evaluates a configurable number of consecutive observations for each entity. The initial setting is a **20-request window (N = 20)** that advances by **five newly observed requests (stride S = 5)**. The examples below show two adjacent assessments; their overlap is deliberate.

Request blocks	1–5	6–10	11–15	16–20	21–25
Window A requests 1–20	**IN**	**IN**	**IN**	**IN**	—
Window B requests 6–25	—	**IN**	**IN**	**IN**	**IN**

Figure 2. Editable Sliding Window diagram. Window A and Window B share 15 requests. The five new requests cause another assessment when the stride boundary is reached.

Why request count alone is not enough

Twenty requests can arrive in seconds during a high-volume scan or over many minutes during low-rate probing. For this reason, a short time horizon limits the ordered input to recent observations, and a longer history summarizes earlier activity. This lets the model inspect local sequence changes without losing every signal of extended persistence.

Control	Initial value	Interpretation
Window N	20 requests	Maximum number of recent ordered observations provided to a decision.
Stride S	5 requests	Decision is revisited when request count reaches the next stride boundary.
Short horizon	300 seconds	Exclude earlier observations from the ordered short-term window.
History horizon	1,800 seconds	Calculate bounded historical statistics for the same entity.
First event / gap	Additional triggers	Partial windows are permitted; no full-window wait is required.

Key implementation details

Causal input. Events are processed in timestamp order. At decision time, the tool cannot see later requests. A new observation after a short-horizon gap can trigger another assessment, but there is no idle-time timer or final end-of-file flush.

Identity. A trusted, service-scoped actor ID is preferred. When it is unavailable, a source-IP fallback is marked explicitly; NAT can merge legitimate users, and proxy rotation can split one actor. ASN enrichment is not included in this version.

Interpretation. A sequence such as “404 → new endpoint → 403 → another endpoint” is an observed temporal association. It does not, by itself, prove that the response caused the following request or that the actor used AI.

Window-size trade-off

Small N may reveal immediate retries, while larger N provides more history across attack phases but also increases input size and irrelevant context. These are qualitative expectations, not measured results. The IPSJ evaluation will isolate the effect of N while holding stride and time horizons fixed.

4. Reading the interactive dashboard

The Arsenal demonstration emphasizes what an analyst can actually inspect. A local browser shows the selected scenario, the configured window parameters, each assessment, and the exact minimized observations corresponding to that assessment. This helps reviewers distinguish the tool's factual evidence from the model's classification.

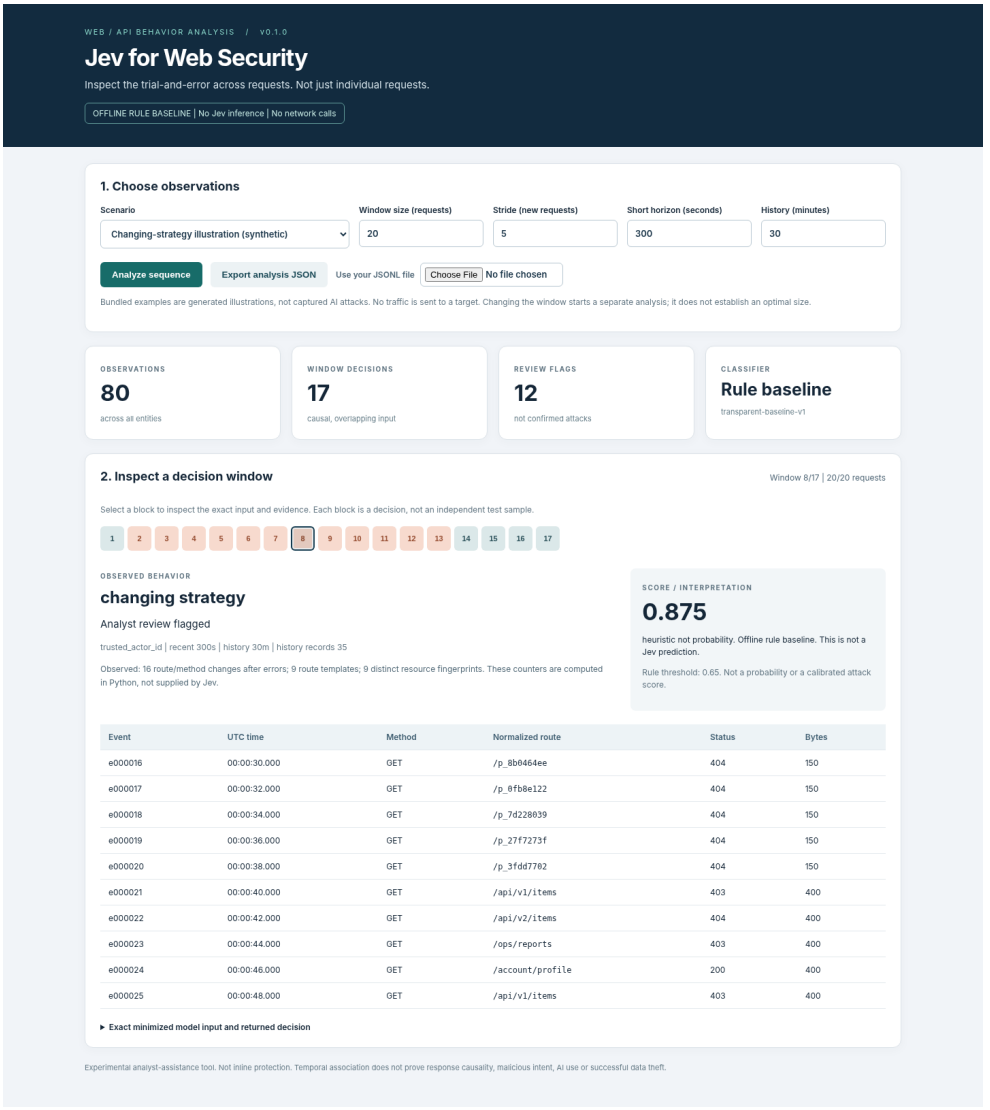


Figure 3. Current local dashboard in offline-rule mode. Screenshot reflects generated demonstration data, not a measured Jev result.

Interface area	How to use it
Scenario + controls	Choose a generated trace; change N, stride, and time horizons; select Analyze sequence.
Window decisions	Select a numbered block to see a particular assessment; blocks are not independent samples.
Behavior and score	Review the label and clearly identified heuristic score or Jev result.
Evidence / export	Inspect the exact observations and export a JSON file for verification.

5. Reproduce the demonstration

The prototype requires Python 3.10 or later and no third-party Python packages for the offline mode. It serves a local dashboard and does not send network requests to a target application. The tool only reads prepared, authorized logs; it does not contain an attack engine.

Quick start: offline baseline

Extract the GitHub-ready ZIP and open a terminal in the folder containing **run.py**. Run:

```
python run.py
```

On macOS/Linux, use **python3 run.py**. On Windows, **START_WINDOWS.bat** is also provided. If the browser does not open automatically, navigate to **http://127.0.0.1:8765/**. Select a scenario, click **Analyze sequence**, inspect one window, and export the JSON result.

Analyze an existing prepared JSONL file

```
python run.py analyze --input examples/adaptive_illustration.jsonl
python -m unittest discover -s tests -v
```

The CLI writes to **outputs/result.json** unless an alternative output is specified. Run tests separately to verify parser, window semantics, and backend contracts.

Enable Jev API mode

After obtaining access and credentials from TypeSafe, stop the offline server and restart it in Jev mode. Supply the key only through the local terminal prompt; do not place it in GitHub, a browser, or the sample logs.

```
python run.py serve --engine jev --ask-key
```

Reload the browser, verify the **JEV API MODE** banner, and enable the explicit external-API consent control before analysis. The default model ID in this release is **jev-1.14.0**. The API call may incur cost. Failed Jev calls are reported as errors; the tool never substitutes rule output while labeling it as Jev inference.

6. Integrating ModSecurity without changing its enforcement path

The supplied adapter accepts a documented subset of ModSecurity v3 JSON Lines, including transaction request method/URI, response status, client IP, and alert messages. It is a format adapter—not a verified live connector. In particular, the tool does not install ModSecurity rules, block requests, or run inside the WAF transaction path.

For meaningful behavioral detection, complete Web access metadata should remain the primary input. A RelevantOnly ModSecurity audit configuration can omit ordinary responses, including seemingly benign HTTP 200 transactions that matter to request-series analysis. WAF alerts should therefore augment the access history, not replace it.

Files a reviewer will use

File	Purpose
run.py	Starts the local dashboard and CLI analysis.
examples/adaptive_illustration.jsonl	Generated trace for the changing-strategy demonstration.
docs/INPUT_FORMAT.md	The accepted JSONL fields, data constraints, and limits.
docs/WINDOWS.md	Exact window-trigger and horizon semantics.
docs/STATUS.md	What has and has not been verified in this release.

7. Demonstration narrative for Black Hat Arsenal

The proposed live demonstration replays three controlled activities against the same analysis interface. The objective is not to assert that every AI-agent run is detectable, but to show how the tool makes the sequence of decisions visible and lets an analyst compare the effects of different window sizes.

Scenario	What the reviewer sees
Legitimate interaction	Normal navigation and authorized API access without a strong persistent-probing pattern.
Conventional scripted scan	Repeated requests and errors, often with limited response-dependent adaptation.
Adaptive exploration illustration	A generated sequence that changes routes or methods after failed responses, prompting a different behavioral review label.

The shipped scenarios are labeled **synthetic illustrations**. They are not a public real-world AI-attack dataset and must not be used as evidence of accuracy. The reviewer can modify N and replay the same trace to observe how the decisions and evidence change.

8. Evaluation boundaries and current readiness

The tool is intentionally conservative about what it has demonstrated. Its role for Arsenal is an inspectable functional prototype. A separate IPSJ research submission will evaluate precision, recall, false positives, detection latency, cross-application generalization, and the contribution of window length using appropriately labeled data.

Status	Current release
Implemented	JSONL parser; session/window logic; deterministic feature extraction; offline rule mode; Jev client; local dashboard; JSON export; ModSecurity JSON subset.
Verified locally	28 Python automated tests, UI interaction checks, and local HTTP smoke checks in the preparation environment.
Not verified yet	Credentialed Jev API execution; Windows/macOS operation; live WAF deployment; real agent-attack accuracy or production throughput.
Security constraints	Read-only localhost application; no active scan or inline blocking; request bodies, headers and query strings excluded by default.

Why reviewers may find the tool useful

The contribution is a small, reproducible behavioral analysis layer that focuses attention on **how a sequence changes** rather than only whether a particular request matches a malicious signature. The key design knobs—window size, stride, and historical context—are visible and tunable, allowing both practical inspection and a disciplined subsequent study of early detection versus false alarms.

Documentation and references

Repository documentation: README.md, docs/WINDOWS.md, docs/INPUT_FORMAT.md, docs/ARCHITECTURE.md, docs/STATUS.md. Official API: docs.typesafe.ai/api. ModSecurity reference: GitHub OWASP ModSecurity Reference Manual v3.x.

Independent research software under MIT license. The hosted Jev API has separate terms.